Тема 7: Процеси и комуникация между тях в операционната система.

ТЕОРИЯ

1. Процес — основна абстракция на ОС

Процес = програма в хода на нейното изпълнение. Абстрактна машина за обработка на информация.

Функционални изисквания:

- Изолация — всеки процес има собствено адресно пространство
- Комуникация — споделени ресурси, файлови дескриптори, памет
- Конкурентност — илюзия за едновременно изпълнение на много процеси

Time-sharing: процесите редуват кратки кванти процесорно време. Превключването се управлява от **task scheduler** чрез timer interrupt (хардуерен таймер).

Два вида чакане на процес:

- **Blocking wait** — чака данни от друг процес/устройство (доброволно)
- **Time-interrupt** — принудително спиране от таймера

> Резултат: редът на изпълнение на инструкции между процеси е **непредсказуем**.

---

2. Комуникационни канали

Комуникационен канал = абстракция за обмен на данни между процеси (предоставя се от ОС).

Пример: `open(file)` → ОС връща **файлов дескриптор** (неотрицателно цяло число) — уникален ID на канала.

---

3. Споделена памет и Race Condition

Споделена памет — регион, достъпен от няколко процеса едновременно.

Race Condition (съревнование за ресурси): резултатът зависи от случайния ред на изпълнение на инструкциите на конкурентните процеси → **недетерминистично поведение**.

Критична секция (Critical Region): частта от кода, в която процесът работи с общия ресурс.

Изискване за коректност: в даден момент **само един процес** може да е в своята критична секция за даден ресурс.

---

4. Хардуерна защита — Spinlock

Spinlock = синхронизационен механизъм от ниско ниво с **активно чакане (busy waiting)**.

Ползва атомарна инструкция **TSL (Test and Set Lock)**:

```
lock:
    TSL R, turn      // атомарно: R = turn; turn = 1
    CMP R, #0        // ако turn беше 0 → можем да влезем
    JNZ lock         // иначе → пак опитай (spin)
    ; --- критична секция ---
unlock:
    MOV turn, #0     // освободи заключването
    RET
```

Предимства: бърз при кратки критични секции, прост.
Недостатъци: губи процесорно време при дълго чакане (busy waiting); не е подходящ при дълги критични секции.

---

5. Семафор — синхронизация от високо ниво

Семафор (Дийкстра) — структура, която **приспива** процеси вместо да ги кара да завъртат цикъл.

Структура:

- `counter` — цяло число (не се чете директно от процесите)

- `list` — списък на блокираните процеси

Методи:

```
init(n):     counter = n

wait(s):     s.counter--
             if s.counter < 0:
                 добави текущия процес в s.list
                 block()   // приспи процеса

signal(s):   s.counter++
             if s.counter <= 0:
                 събуди един процес от s.list
                 wakeup(process)
```

Видове:

- **Двоичен семафор** (mutex): init(1) — взаимно изключване
- **Бройков семафор**: init(n) — брои налични ресурси (n > 1)

Сигнален семафор: init(0) — синхронизация на събитие (A сигнализира, B чака)

---

6. Block / Wakeup

block() — прекратява изпълнението на текущия процес, поставя го в опашка за чакане.
wakeup(p) — събужда процес p и го поставя в опашката за изпълнение.

> Реализирани от ОС. Не губят процесорно време (за разлика от spinlock).

---

7. Реализация на Pipe (комуникационна тръба) чрез семафори

Pipe = споделен буфер от n елемента. Producer добавя, Consumer взима.

```
mutex_P  = Semaphore(1)   // взаимно изключване при запис
mutex_C  = Semaphore(1)   // взаимно изключване при четене
items    = Semaphore(0)   // брой налични елементи за четене
spaces   = Semaphore(n)   // брой свободни места за запис

Producer:
    event = waitForEvent()
    spaces.wait()          // чакай свободно място
    mutex_P.wait()         // влез в критична секция
    buffer.add(event)
    mutex_P.signal()       // излез от критична секция
    items.signal()         // сигнализирай нов елемент

Consumer:
    items.wait()           // чакай наличен елемент
    mutex_C.wait()         // влез в критична секция
    event = buffer.get()
    mutex_C.signal()       // излез от критична секция
    spaces.signal()        // сигнализирай свободно място
    event.process()
```

КАК СЕ РЕШАВАТ ЗАДАЧИ

> Тази тема е **Task 4** на практическия изпит (появява се в 2015–2024). Имплементация на синхронизация чрез семафори.

Метод: Как се поставят семафорите

Правило: Ако инструкция A трябва да се изпълни преди инструкция B:

1. Дефинирай сигнален семафор `s = Semaphore(0)`
2. Постави `signal(s)` **след A** (в процеса на A)
3. Постави `wait(s)` **преди B** (в процеса на B)

Правило за взаимно изключване: Ако никой от два блока не трябва да е едновременно активен:

1. `mutex = Semaphore(1)`
2. Обгради всеки блок: `mutex.wait()` ... `mutex.signal()`

Задача 3 от конспекта: p1 → q2 → p3

```
Изисквания: p1 преди q2; q2 преди p3
```

Решение:

```
s1 = Semaphore(0)   // p1 → q2
s2 = Semaphore(0)   // q2 → p3

process P:          process Q:
    p1                  q1
    signal(s1)          wait(s1)
    p2                  q2
    wait(s2)            signal(s2)
    p3                  q3
```

Проверка: p1 → signal(s1) → Q може да мине wait(s1) → q2 → signal(s2) → P може да мине wait(s2) → p3. ✓

Задача 2 от конспекта: тройна синхронизация P, Q, R

```
Изисквания:
(а) p1 преди q2 и r2
(б) ако q2 преди r2 → q3 преди r2
(в) ако r2 преди q2 → r3 преди q2
```

Анализ на (б) и (в): Условия (б) и (в) заедно означават: Q и R не могат едновременно да са в секцията [q2,q3] / [r2,r3]. Изпълнението им е взаимно изключващо се. → **mutex** семафор.

Решение:

```
s_pq  = Semaphore(0)   // (а): p1 → q2
s_pr  = Semaphore(0)   // (а): p1 → r2
mutex = Semaphore(1)   // (б)(в): q2..q3 и r2..r3 са взаимно изключващи се

process P:
    p1
    signal(s_pq)        // отключи Q за q2
    signal(s_pr)        // отключи R за r2
    p2
    p3

process Q:
    q1
    wait(s_pq)          // чакай p1
    wait(mutex)         // влез в "средна секция"
    q2
    q3
    signal(mutex)       // освободи "средна секция"

process R:
    r1
    wait(s_pr)          // чакай p1
    wait(mutex)         // влез в "средна секция"
    r2
    r3
    signal(mutex)       // освободи "средна секция"
```

Проверка:

- (а): Q не може да мине wait(s_pq) преди P изпълни p1. ✓
- (б): Ако Q взема mutex преди R: Q прави q2, q3, освобождава mutex → тогава R може да взема mutex и прави r2. Значи q3 преди r2. ✓
- (в): Симетрично — ако R взема mutex преди Q. ✓

Задача 1 от конспекта: P и N копия на Q

```
Изисквания:
- p1 преди q2 на всяко копие на Q
- след p1, ВСИЧКИ копия на Q да завършат (q3)
```

Решение (N = брой копия на Q, известен предварително):

```
start = Semaphore(0)   // p1 → q2 (N пъти)
done  = Semaphore(0)   // всяко Q сигнализира при завършване

process P:
    p1
    for i in 1..N:
        signal(start)  // отключи всяко Q
    for i in 1..N:
        wait(done)     // изчакай всяко Q да завърши
    p2
    p3

process Q (всяко копие):
    q1
    wait(start)        // чакай p1
    q2
    q3
    signal(done)       // уведоми P, че сме свършили
```

Проверка:

- p1 се изпълнява преди signal(start) → Q не може да мине wait(start) преди p1. ✓
- P изчаква N пъти на done → p2 се изпълнява след всички Q са завършили q3. ✓

Обобщен шаблон за задачи за синхронизация

Стъпка 1: За всяко „A преди B" изискване → `s = Semaphore(0)`, signal след A, wait преди B.

Стъпка 2: За „не едновременно" изискване (взаимно изключване на блокове) → `mutex = Semaphore(1)`, обгради всеки блок с wait/signal.

Стъпка 3: За „изчакай N събития" → `s = Semaphore(0)`, N пъти signal, N пъти wait.

Стъпка 4: Провери за **deadlock** — нняма цикъл от взаимни изчаквания.

Стъпка 5: Провери всяко изискване чрез разиграване на сценарий.

Бърза справка

Конструкция	Употреба
`s = Semaphore(0)` + signal/wait	Сигнално — A преди B
`s = Semaphore(1)` + wait/signal	Mutex — взаимно изключване
`s = Semaphore(n)`	Бройков — n налични ресурса
`for N: signal` + `for N: wait`	Изчакай N процеса
signal **след** събитието	Не преди — иначе B може да мине веднага
wait **преди** зависимото	Не след — иначе твърдението е безсмислено